CONTENTS

TraceWeave-Core — Anticipated Professor Questions (Non-Technical Panel)

1. “Explain it like I know nothing” questions

2. “Prove you’re not making this up” questions

3. “Why should I care / so what” questions

4. “Explain your process” questions

5. Questions that sound stupid but are actually fair (be ready, don’t scoff)

6. Scope, honesty, and “gotcha” questions

Quick-reference cheat sheet (say these if you blank on stage)

TraceWeave-Core — Anticipated Professor Questions (Non-Technical Panel)

Plain-English answers you can say out loud without sounding like you're reading a spec. Organized by the *kind* of question, because cocky professors tend to attack from a predictable set of angles: "explain it like I'm five," "prove this isn't fake," "why does this matter," and the classic questions that sound dumb but are actually testing whether you understand your own project.

---

1. "Explain it like I know nothing" questions

Q1. In one sentence, what does your project actually do? We check how often tools that read code *without running it* get confused about which function calls which — and we measure exactly how often that happens on real, popular Python projects.

Q2. What is a "call graph"? Use an analogy. Think of a company org chart, but for code — it shows "this piece of code calls that piece of code." A call graph is a map of who talks to whom inside a program.

Q3. What does "static" mean here? Why do you keep saying that word? Static means "reading the code like a book, without running it" — the way you'd read a recipe without cooking. The opposite, "dynamic" or "runtime," means actually running the program and watching what really happens, like actually cooking the recipe and seeing what you end up with.

Q4. So what's the actual problem you found? Some Python code decides which function to call *while it's running*, not while you're reading it — like a restaurant where the waiter decides your order based on a dice roll instead of the menu. Tools that only read the code (never run it) can't see those decisions. We measure how much of that they miss.

CONFIDENTIAL

Q5. Why does this matter to anyone outside your classroom? AI coding assistants (like GitHub Copilot-style tools) increasingly use these code maps to understand a codebase before suggesting changes. If the map is wrong or incomplete, the AI's "understanding" of the code is quietly wrong too — and nobody had actually measured how wrong, until this.

Q6. Is this an AI project? No — that's the point. We are not training or testing an AI model. We're measuring a property of *programs themselves* (how much real behavior a code-reading tool misses). The result doesn't depend on which AI model exists this year, so it won't go stale.

---

2. "Prove you're not making this up" questions

Q7. How do you know your numbers aren't just wrong or made up? We wrote down our rules — what counts as a "miss," what counts as a fair comparison — *before* we ran any code, and we're not allowed to change those rules after seeing the results. That's the same discipline clinical drug trials use so researchers can't quietly move the goalposts.

Q8. What if you just got a fluke result on one weird piece of code? We're not testing one program — we're running this across hundreds of real, popular open-source Python projects (over 550 candidates so far), grouped by how "dynamic" their coding style is, so we can see whether the pattern holds broadly or only in edge cases.

Q9. Couldn't your own extraction tool just be bad, and that's why it "misses" things? We tested with two completely independent tools built by different teams, not just our own code, so one buggy tool can't fake the result. If both agree on a miss, that's much stronger evidence than either one alone.

Q10. What if the code just never runs the "missing" part — maybe it's dead code? Good catch — that's exactly why we only count something as evidence if we *actually watched it execute*. We don't guess; we run the program for real and record what happened, so a miss is a call that genuinely fired, not a hypothetical one.

CONFIDENTIAL

Q11. How do you know the tool didn't just fail to run properly? We log every attempt in detail — what version of Python, what packages were installed, whether it crashed — and we throw out or clearly label anything that didn't run cleanly rather than quietly excluding it. Failed runs are also useful information: they get reported as failures, not hidden.

Q12. Is this reproducible? Can someone else redo your experiment? Yes — that's actually one of our project's outputs, not an afterthought. We're releasing exactly which code versions we tested, exactly what "run" of the program we used, and the tools we used, so someone else could plug in the same pieces and get the same answer.

---

3. "Why should I care / so what" questions

Q13. Say your number comes back small. Wasn't this a waste of time? No — a small number is still useful, because right now *nobody knows the number at all*, small or large. If it turns out static tools are mostly fine except in one or two specific coding patterns, that's a genuinely useful, publishable finding: it tells people exactly what to fix instead of "something is wrong somewhere."

Q14. What happens if your number comes back huge? Then it's evidence that a lot of tools people trust today (documentation generators, AI coding assistants, refactoring tools) are working off an incomplete picture of the code, and it points to exactly which kind of code pattern is causing the blind spot.

Q15. Who actually uses this result — a real person, not "the field"? Anyone building a tool that reads code automatically — an AI code assistant, an automated refactoring tool, a security scanner that maps how data flows through a program. They all lean on a call graph internally, and this tells them how much to trust it.

Q16. Isn't this just an academic exercise with no real-world use? We also release the actual test data — the programs, the "correct answer" call graphs, the ready-to-run test cases — as a public benchmark, so other researchers can use it to test their own tools later without redoing our work. That's a reusable artifact, not just a one-time number.

CONFIDENTIAL

4. “Explain your process” questions

Q17. Walk me through your method in plain terms. Four steps: (1) grab a bunch of real Python projects from GitHub, (2) get each one to actually run — install it, run its tests — inside a controlled sandbox, (3) have humans record a few realistic ways to actually *use* each program, (4) run the program for real while watching every function call, and separately ask a code-reading tool to guess the call graph without running it. Then compare the two.

Q18. Why do you need real humans in the loop? Can’t this all be automated? A machine can’t reliably tell “this paragraph in a README is a working example command” from “this is just decorative text.” Reading documentation and figuring out how to actually exercise a program the way a real developer would is a judgment call — we automated everything that’s mechanical and kept humans only where judgment is genuinely needed.

Q19. Why do you separate projects into “easy,” “medium,” “hard” difficulty buckets? Because the phenomenon we’re measuring — code deciding calls at runtime — happens a lot more in some coding styles than others. If we tested only simple projects, we might falsely conclude the problem barely exists; grouping by how much of that dynamic style a project uses lets us see the real pattern instead of an average that hides it.

Q20. Why do you run everything inside those container/sandbox things instead of just on your laptop? Two reasons anyone can follow: it keeps every test running under identical, controlled conditions so results are comparable across hundreds of different programs, and it protects our own machine — we’re running code from strangers on the internet, and a sandbox contains anything that misbehaves.

---

5. Questions that sound stupid but are actually fair (be ready, don’t scoff)

Q21. Isn’t “call graph” just a flowchart? Why the fancy name? Close enough as an intuition — it is a kind of diagram of “what leads to what.” The difference is a flowchart usually shows the *logic* of one program’s steps; a call graph specifically shows which

functions invoke which other functions, which is what tools use to understand a codebase's structure.

Q22. Why Python? Why not a "harder" language? Python is exactly where this problem is worst, which is why it's the interesting case — it makes it very easy to decide function calls at runtime (compared to something like Java or C), and it's also the most common language in real-world codebases and in the tools we're worried about (AI coding assistants are trained heavily on Python).

Q23. Did you write this code yourselves or is it copy-pasted from somewhere? Every script and configuration file here was written or configured by us for this project — we do use two existing, published research tools as one part of the comparison (that's intentional and disclosed, not something we're hiding), but the pipeline connecting everything is ours.

Q24. Can I just run this on my laptop right now and see it work? Yes — we have a small live demo: one tiny example program with a deliberately "hidden" function call, shown side-by-side as the static tool's guess versus what actually happens when it runs. It takes under 3 seconds.

Q25. What if the internet goes down during your demo? We've planned for that — we keep a recorded, pre-run copy of the exact same demo output as a backup, so a dead Wi-Fi connection doesn't stop the presentation.

Q26. Are you sure this isn't just ChatGPT writing bad code and you're measuring that? No language model is involved in producing or grading the actual result — we don't use any AI "judge" anywhere in the measurement itself. It's plain code analysis on one side and plain code execution on the other; both sides are deterministic, not AI-generated.

Q27. What's the difference between this and just running "pip install" and seeing if it works? That's actually one small piece of our pipeline (step 2, "does it even run") — but the interesting work happens after that: watching *what the program does while it's running* and comparing that to what a reader-only tool predicted.

Q28. Why do you need so many projects? Isn't 10 examples enough to prove a point? A pattern seen in 10 hand-picked examples could just be us cherry-picking favorable cases. Testing hundreds of projects we didn't choose for convenience is what

CONFIDENTIAL

turns “here’s an example” into “here’s a measurement.”

---

6. Scope, honesty, and “gotcha” questions

Q29. What does your project NOT do? What are you leaving out? We’re not fixing the problem or building a better tool that closes the gap — we’re only measuring how big the gap is and where it concentrates. We’re also not testing any other programming language, and we’re not doing anything with AI model training or comparison.

Q30. Is this finished? What’s left? The measurement pipeline itself is built and tested; what’s left is running it at full scale across all the collected projects and getting the final headline number — that’s the next phase, not a sign anything is behind schedule.

Q31. What happens if a project you tested breaks or misbehaves during testing? We record it as a failure with a specific reason (didn’t install, crashed, timed out) rather than silently dropping it — a project that fails to run is still useful information about how hard code is to analyze in the real world, so it’s never just discarded.

Q32. Someone could say your test programs are unrepresentative — how do you defend that? We pulled real, independently popular open-source projects from GitHub rather than writing our own toy examples, specifically so we can’t be accused of designing test cases to get the answer we wanted.

Q33. Did your guide/advisor approve this approach, or is this just your own idea? Every major decision — what counts as a fair test, which thresholds to use, when something is “finished” — is reviewed and signed off before we lock it in, precisely so it isn’t just one student’s unchecked judgment call.

Quick-reference cheat sheet (say these if you blank on stage)

- **What:** Measuring how much of real code’s actual behavior a “read-only” code analysis tool misses.
- **Why it’s hard:** Python lets code decide function calls while running, which reading-only tools can’t see.

- **Why it's trustworthy:** Rules were fixed before results existed; two independent tools used; hundreds of real projects, not hand-picked ones; nothing silently thrown away.
- **Why it matters:** AI coding tools and code-analysis tools quietly rely on these maps being right; nobody had measured how wrong they can be.
- **What ships:** A number (how much is missed), a reusable public test set other researchers can use, and a report on which coding patterns cause the problem.

CONFIDENTIAL